

Recursion

Chapter 5: A. Drozdek

Objectives

Discuss the following topics:

- Recursive Definitions
- Method Calls and Recursion Implementation
- Anatomy of a Recursive Call
- Nested Recursion

Recursive Definitions

- **Recursive definitions** are programming concepts that define themselves
- A recursive definition consists of two parts:
 - The **anchor** or **ground case**, the basic elements that are the building blocks of all other elements of the set
 - Rules that allow for the construction of new objects out of basic elements or objects that have already been constructed

Recursive Definitions (continued)

- Recursive definitions serve two purposes:
 - **Generating** new elements
 - **Testing** whether an element belongs to a set
- Recursive definitions are frequently used to define functions and sequences of numbers

Method Calls and Recursion Implementation

- Activation records contain the following:
 - Values for all parameters to the method, location of the first cell if an array is passed or a variable is passed by reference, and copies of all other data items
 - Local (automatic) variables that can be stored elsewhere
 - The return address to resume control by the caller, the address of the caller's instruction immediately following the call

Method Calls and Recursion Implementation (continued)

- A dynamic link, which is a pointer to the caller's activation record
- The returned value for a method not declared as void

The factorial function

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (n-1) \cdot n$$

↑
??

$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ (base case, anchor)} \\ n \cdot (n-1)! & \text{if } n > 0 \text{ (inductive step)} \end{cases}$$

Application of the definition of $n!$

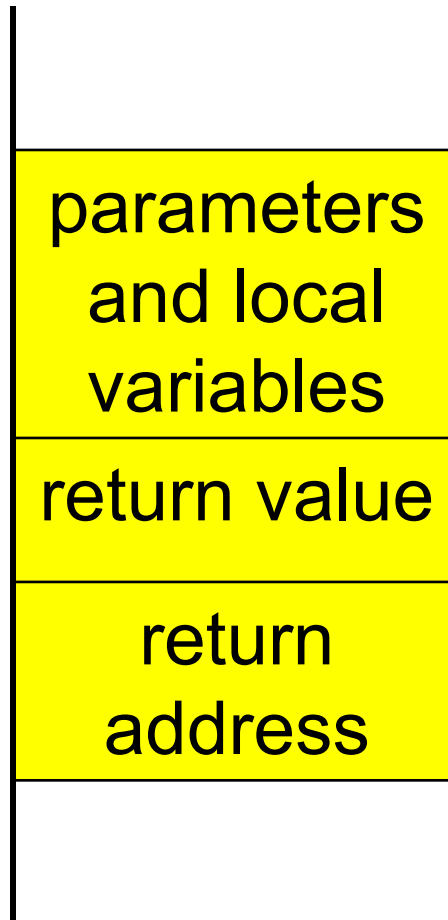
$$\begin{array}{l} 3! = 3 \cdot 2! = ? = 3 \cdot 2 = 6 \\ \downarrow \\ 2! = 2 \cdot 1! = 2 \cdot 1 = 2 \\ \downarrow \\ 1! = 1 \cdot 0! = 1 \cdot 1 = 1 \\ \downarrow \\ 0! = 1 \end{array}$$

The diagram illustrates the recursive calculation of factorials. It shows a sequence of equations: $3! = 3 \cdot 2! = ? = 3 \cdot 2 = 6$, $2! = 2 \cdot 1! = 2 \cdot 1 = 2$, $1! = 1 \cdot 0! = 1 \cdot 1 = 1$, and $0! = 1$. Arrows indicate the flow of the calculation: a vertical arrow from $3!$ to $2!$, a vertical arrow from $2!$ to $1!$, a vertical arrow from $1!$ to $0!$, and diagonal arrows pointing upwards from $0!$ to $1!$ and from $2!$ to $3!$.

Implementation of the factorial function

```
long factorial(long n) {  
    if (n == 0)  
        return 1;  
    else return n * factorial(n-1);  
}
```

Stack frame



Executing the factorial method

```
long factorial(long n) {  
    if (n == 0)  
        return 1;  
(10)    else return n * factorial(n-1);  
}  
  
.....  
void f() {  
    .....  
(20)    long m = factorial(3);  
    .....
```

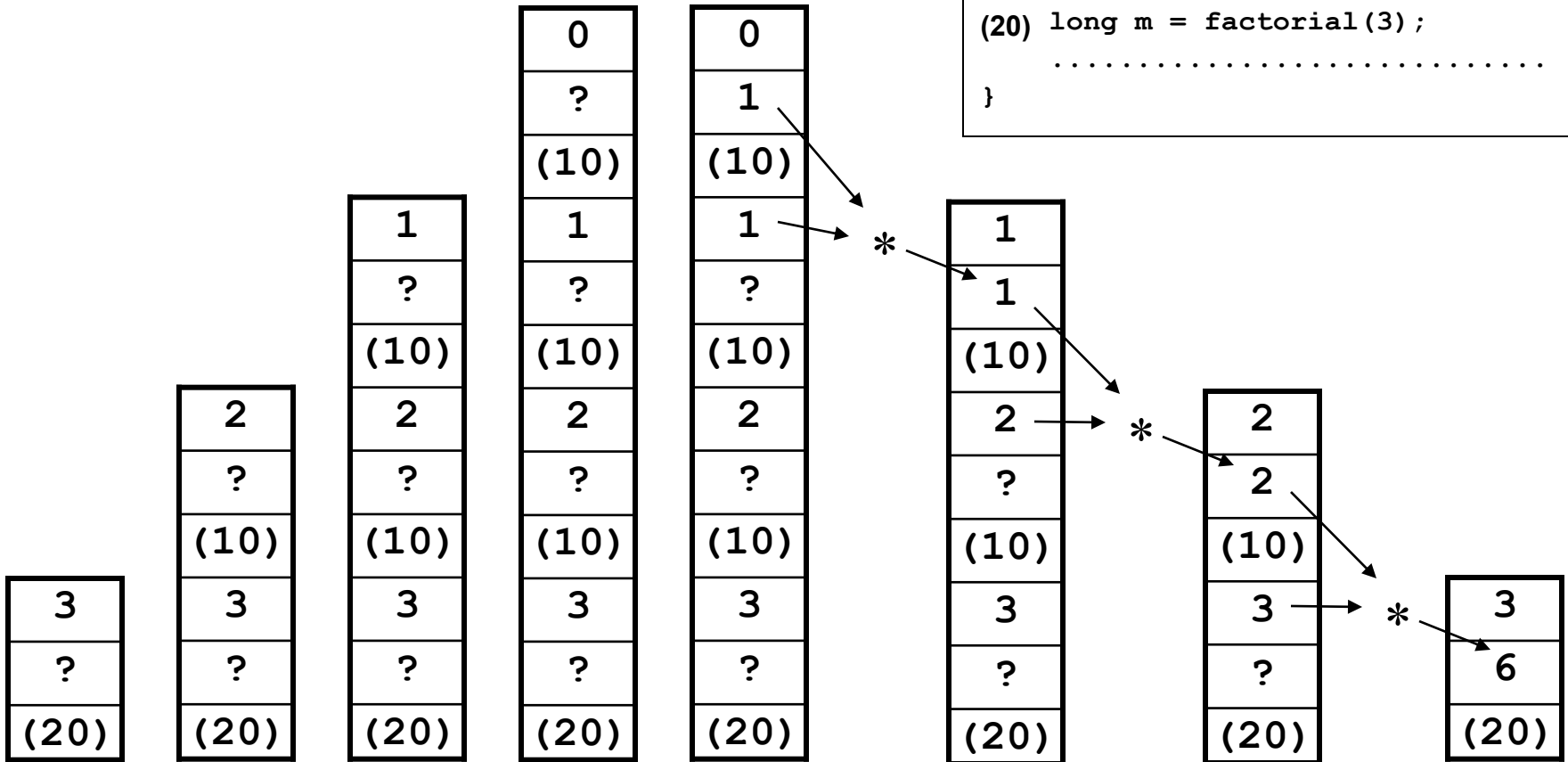
```

long factorial(long n) {
    if (n == 0)
        return 1;
    (10) else return n * factorial(n-1);
}

.....

void f() {
    .....
    (20) long m = factorial(3);
    .....
}

```



Tracing recursion

```
void f(int n) {  
    if (n > 0) {  
        f(n-1);  
        System.out.print(n + " ");  
        f(n-1);  
    }  
}
```

Tracing recursion using indentation

f(1)

 f(0)

 1

 f(0)

output: 1

f(2)

 f(1)

 f(0)

 1

 f(0)

 2

 f(1)

 f(0)

 1

 f(0)

output: 1 2 1

f(3)

 f(2)

 f(1)

 f(0)

 1

 f(0)

 2

 f(1)

 f(0)

 1

 f(0)

 3

 f(2)

 f(1)

 f(0)

 1

 f(0)

 2

 f(1)

 f(0)

 1

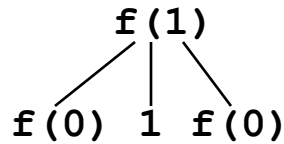
 f(0)

output: 1 2 1 3 1 2 1

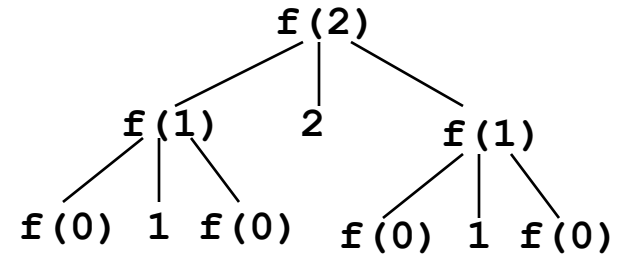
```
void f(int n) {  
    if (n > 0) {  
        f(n-1);  
        System.out.print(n + " ");  
        f(n-1);  
    }  
}
```

Data Structures and Algorithms in Java

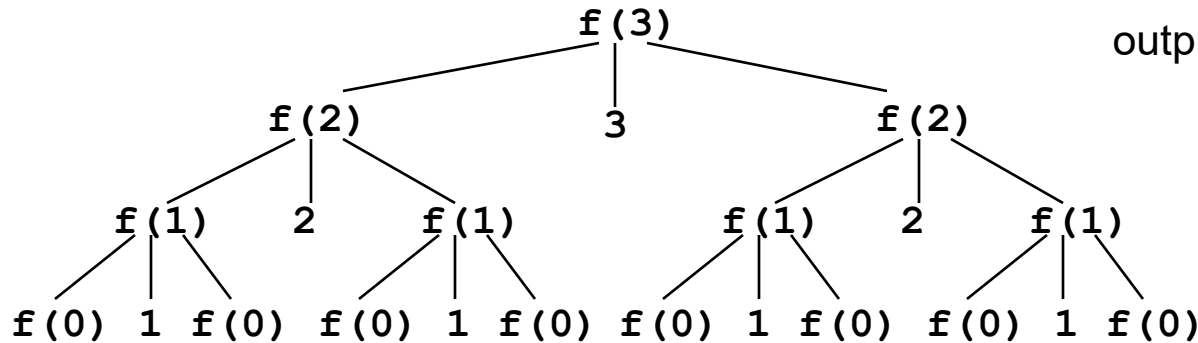
Tracing recursion using tree of calls



output: 1



output: 1 2 1



output: 1 2 1 3 1 2 1

```
void f(int n) {  
    if (n > 0) {  
        f(n-1);  
        System.out.print(n + " ");  
        f(n-1);  
    }  
}
```

Nested Recursion

Nested recursion is not only defined in terms of itself, but also is used as one of the parameters.

$$h(n) = \begin{cases} 0 & \text{if } n = 0 \\ n & \text{if } n > 4 \\ h(2 + h(2n)) & \text{if } n \leq 4 \end{cases}$$

$h(10) ?$

$h(2) ?$

$h(1) ? h(3) ? h(4) ?$

$$h(n) = \begin{cases} 0 & \text{if } n = 0 \\ n & \text{if } n > 4 \\ h(2 + h(2n)) & \text{if } n \leq 4 \end{cases}$$

$$h(n) = \begin{cases} 0 & \text{if } n = 0 \\ n & \text{if } n > 4 \\ h(2 + h(2n)) & \text{if } n \leq 4 \end{cases}$$

$$h(n) = \begin{cases} 0 & \text{if } n = 0 \\ n & \text{if } n > 4 \\ h(2 + h(2n)) & \text{if } n \leq 4 \end{cases}$$

$$h(n) = \begin{cases} 0 & \text{if } n = 0 \\ n & \text{if } n > 4 \\ h(2 + h(2n)) & \text{if } n \leq 4 \end{cases}$$

```
int h( n ) {
    if ( n == 0 ) return 0;
    else if (n > 4) return n;
    return h(2 + h(2 * n) );
}
```


Excessive Recursion

- Using recursion is logically simple and readable.
- However, when recursion is too deep, it
 - slows down the execution time
 - stores more values/instructions on the run-time stack than nonrecursive approach.

$$\text{Fib}(n) = \begin{cases} n & \text{if } n < 2 \\ \text{Fib}(n-2) + \text{Fib}(n-1) & \text{otherwise} \end{cases}$$

Excessive Recursion

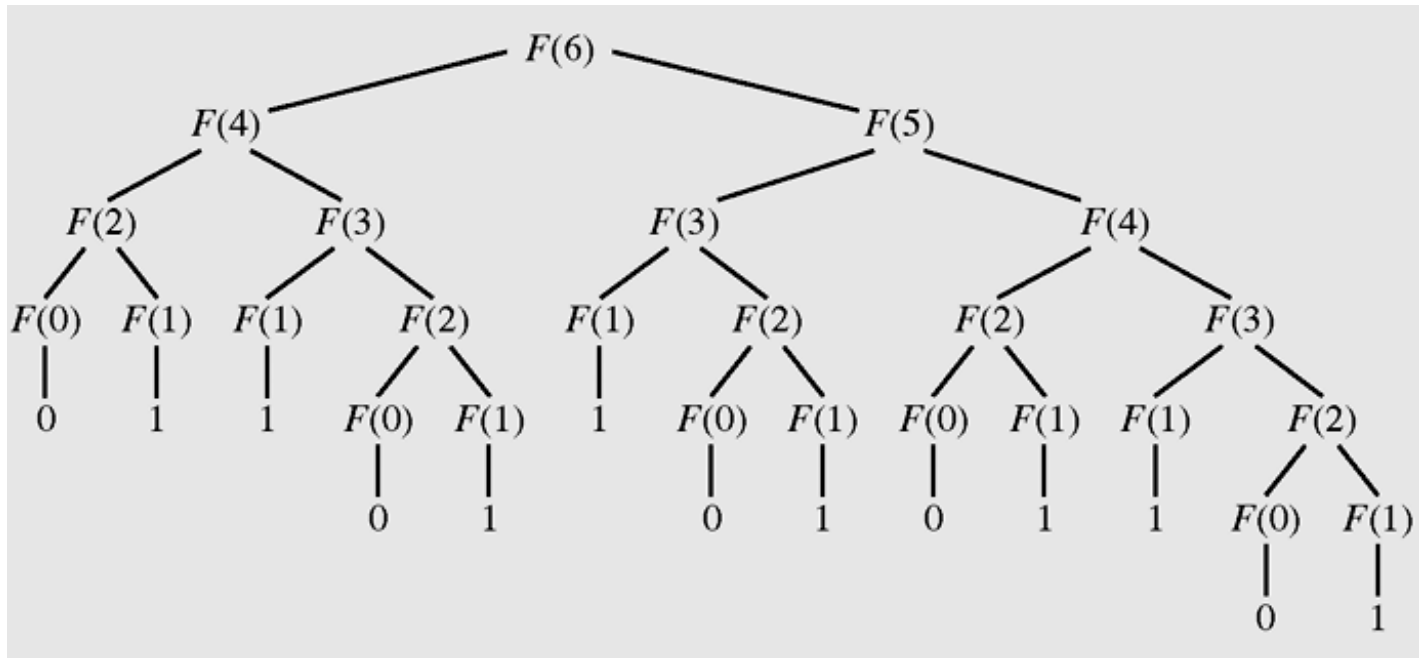


Figure 5-8 The tree of calls for $\text{Fib}(6)$

Excessive Recursion

```
int Fib(int n) {  
    if (n < 2)  
        return n;  
    else return Fib(n-2) + Fib(n-1);  
}
```

Write Fib methods by iterative approach (nonrecursion).

Excessive Recursion (continued)

n	Fib(n+1)	Number of Additions	Number of Calls
6	13	12	25
10	89	88	177
15	987	986	1,973
20	10,946	10,945	21,891
25	121,393	121,392	242,785
30	1,346,269	1,346,268	2,692,537

Figure 5-9 Number of addition operations and number of recursive calls to calculate Fibonacci numbers

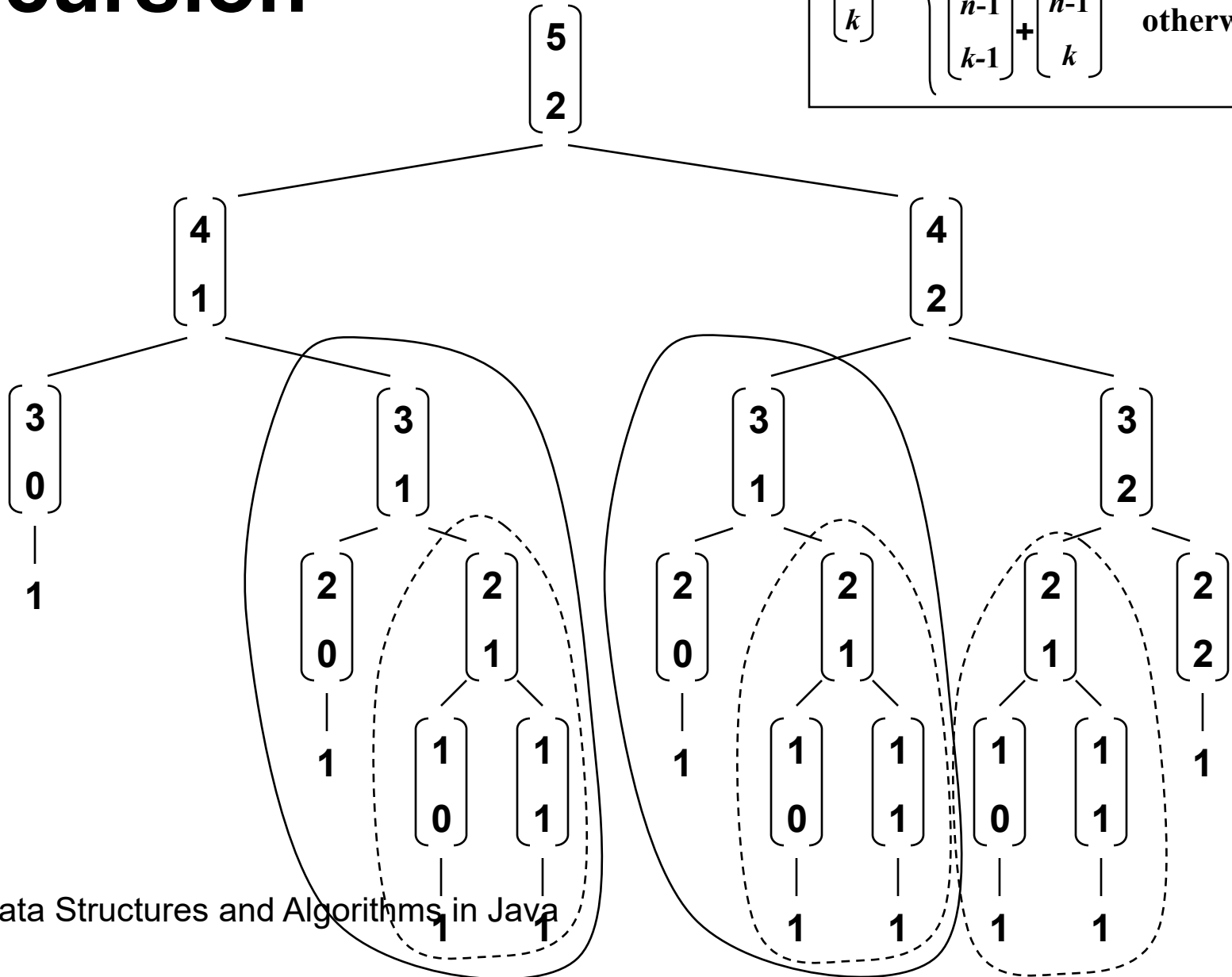
Excessive Recursion (continued)

n	Number of Additions	Assignments	
		Iterative Algorithm	Recursive Algorithm
6	5	15	25
10	9	27	177
15	14	42	1,973
20	19	57	21,891
25	24	72	242,785
30	29	87	2,692,537

Figure 5-10 Comparison of iterative and recursive algorithms for calculating Fibonacci numbers

Excessive recursion

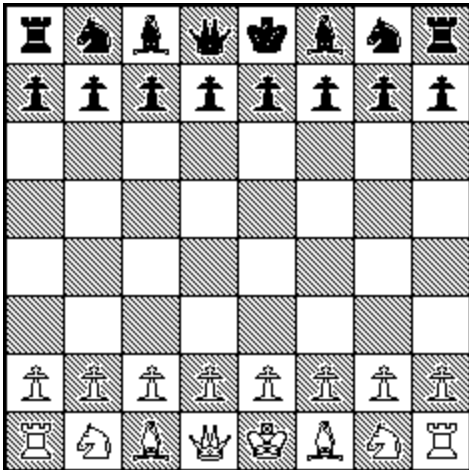
$$\begin{bmatrix} n \\ k \end{bmatrix} = \begin{cases} 1 & \text{if } k = 0 \text{ or } k = n \\ \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} + \begin{bmatrix} n-1 \\ k \end{bmatrix} & \text{otherwise} \end{cases}$$



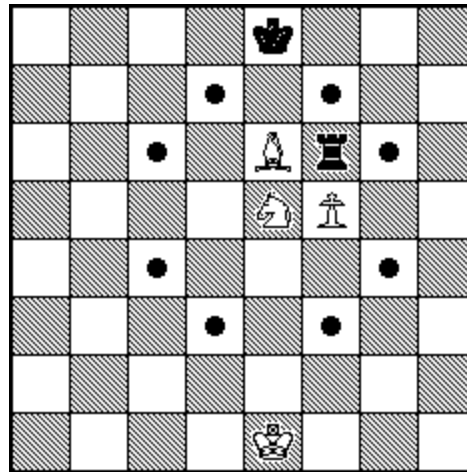
Backtracking

- **Backtracking** is a technique for returning to a given position (e.g., entry point) after trying other avenues that are unsuccessful in solving a particular problem

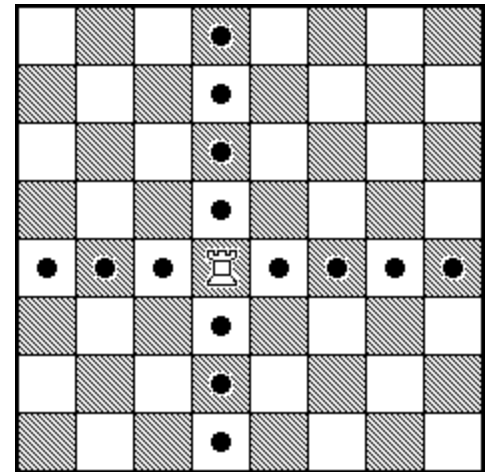
Chess Move



Horse ?

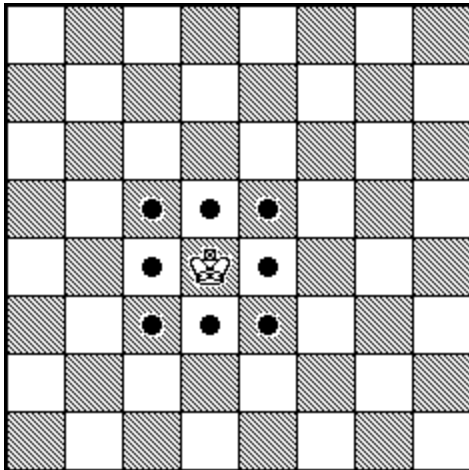


Rook ?

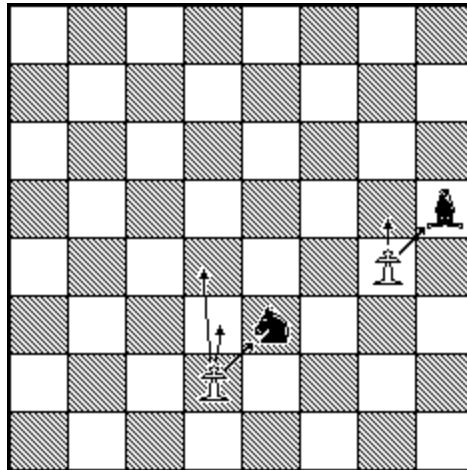


Chess Move

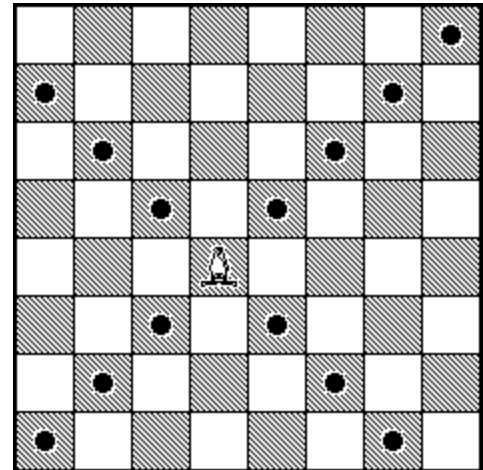
King ?



Pawn ?

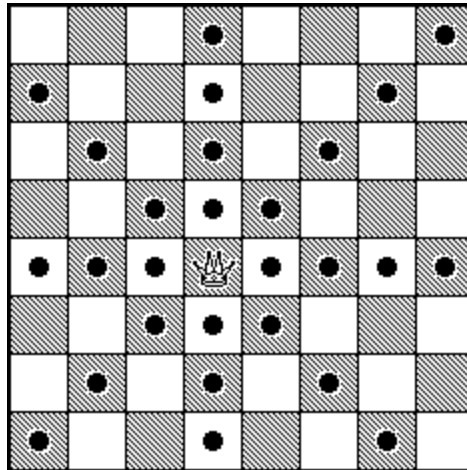


Bishop ?



Queen Move

Queen ?



Backtracking (continued)

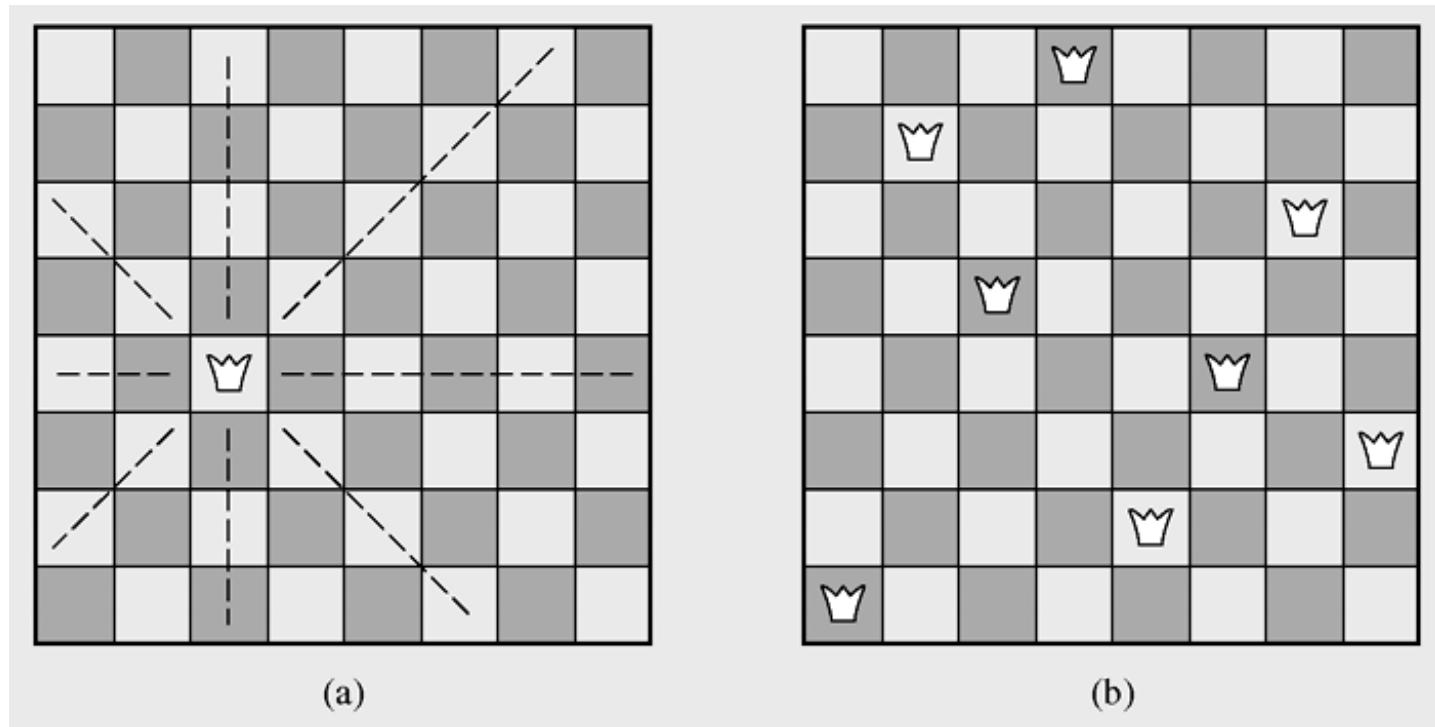


Figure 5-11 The eight queens problem

Eight Queens Problem

```
putQueen(row)
    for every position col on the same row
        if position col is available
            place the next queen in position col;
            if (row < 8)
                putQueen(row+1);
            else print successful path;
                remove the queen from position col;
```

Four Queens Problems

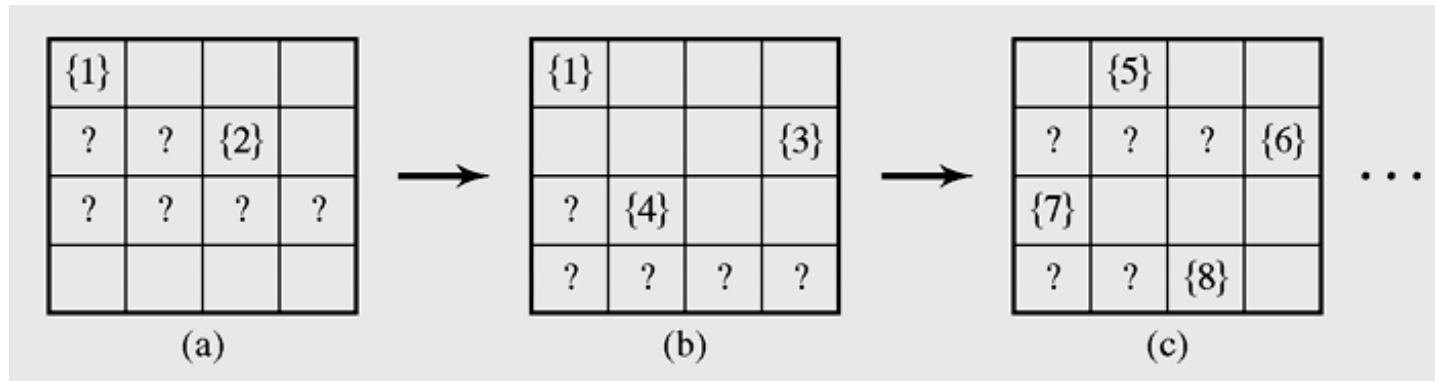


Figure 5-17 Changes to the chessboard leading to the first successful configuration

Summary

- Recursive definitions are programming concepts that define themselves
- Recursive definitions serve two purposes:
 - Generating new elements
 - Testing whether an element belongs to a set
- Recursive definitions are frequently used to define functions and sequences of numbers

Summary (continued)

- Tail recursion is characterized by the use of only one recursive call at the very end of a method implementation.
- Backtracking is a technique for returning to a given position (e.g., entry point) after trying other paths that are unsuccessful in solving a particular problem.